动机：Google论文《Pass@K Policy Optimization: Solving Harder Reinforcement Learning Problems》给出了直接优化Pass@K的方法。但是，没有给出0-1二元奖励下的显式解。此外，也没有与当前主流方法GRPO进行对比。

本人贡献：

- 针对Google的论文的方法，进一步给出了0-1二元奖励下的显式解。
- 建立了GRPO与直接优化Pass@K指标之间的关系，通过实验初步证明了GRPO隐式优化Pass@K。

一、$\mathrm{Pass@k}$的定义

$\mathrm{Pass@k}$是指模型生成k个候选答案中至少有一个正确的概率。

解析定义。假设一个策略生成正确解的概率为p，那么连续生成k个错误解的概率为$(1-p)^k$。因此，生成k个中至少有一个正确解的概率为$1-(1-p)^k$。因此，$\mathrm{Pass@k}$的解析定义为

$$\mathrm{Pass@k} = 1-(1-p)^k \quad (1)$$

经验定义。从策略$p(x;\theta)$中独立同分布地抽取k个样本，这k个样本中至少有一个成功的概率的经验定义为

$$\mathrm{Pass@k} = \mathbb{E}_{x_1,x_2,\ldots,x_k\sim p(x|\theta)}[1-\prod_{i=1}^{k}(1-f(x_i))] \quad (2)$$

其中$f(x_i)\in\{0,1\}$是x_i的正确性标签。

二、$\mathrm{Pass@k}$的朴素有偏估计

一个朴素的想法，直接采用p的无偏估计代入到$\mathrm{Pass@k}$的定义中，是否就是$\mathrm{Pass@k}$的无偏估计了呢？

假设策略对一个问题生成n个解，其中正确解的数量为c，那么概率p的无偏估计为$\hat{p}=\frac{c}{n}$。那么，朴素的估计量为

$$\mathrm{Pass@k_{naive}} = 1-(1-\hat{p})^k \quad (3)$$

要判断这个估计是否是无偏估计，即判断$\mathbb{E}[(1-\hat{p})^k]$是否等于$(1-p)^k$。这里，令$g(x)=(1-x)^k$，由于该函数为凸函数，依据`Jensen's Inequality`有

$$\mathbb{E}[g(\hat{p})] \geq g(E[\hat{p}]) = g(p)$$

显然，$\mathrm{Pass@k_{naive}}$并不是$\mathrm{Pass@k}$的无偏估计。

三、$\mathrm{Pass@k}$的无偏估计

3.1 随机子集视角下的无偏估计

经验定义公式(2)其实就是$\mathrm{Pass@k}$的无偏估计，但是其主要问题是高方差。那么，假如采样$n(>k)$个样本，能够得到一个更好的无偏估计？由于n个样本中的任意包括k个元素的子集期望均为$\mathrm{Pass@k}$。那么不妨假设$\mathcal{I}$是从n个样本中随机采样k个元素的子集，那么

$$\text{Pass@k} = \mathbb{E}_{x_1,x_2,\ldots,x_n\sim p(x|\theta)}\mathbb{E}_{\mathcal{I}}\left[1-\prod_{i\in\mathcal{I}}(1-f(x_i))\right] \quad (4)$$

基于公式(4)可以发现，我们要的无偏估计量就是内层的期望。因此，

$$\begin{aligned}\mathbb{E}_{\mathcal{I}|x_1,x_2,\ldots,x_n}\left[1-\prod_{i\in\mathcal{I}}(1-f(x_i))\right] &= \frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}\subseteq\{1,2,\ldots,n\}} 1-\prod_{i\in\mathcal{I}}(1-f(x_i)) \\ &= 1-\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}\subseteq\{1,2,\ldots,n\}}\prod_{i\in\mathcal{I}}(1-f(x_i)) \\ &= 1-\frac{\binom{n-c}{k}}{\binom{n}{k}} \\ &\equiv \rho(n,c,k)\end{aligned} \quad (5)$$

其中$\sum_{|\mathcal{I}|=k,\mathcal{I}\subseteq\{1,2,\ldots,n\}}\prod_{i\in\mathcal{I}}(1-f(x_i))$当且仅当$k$个元素的子集全错时为1，而共有$\binom{n-c}{k}$个全错的情况。

3.2 基于Rao-Blackwell定理的无偏估计

> **Rao-Blackwell定理**：
>
> 设$\delta(X)$是参数θ的任意无偏估计量，$T(X)$是参数θ的一个充分统计量。
>
> 那么可以定义一个新的估计量$\delta^*(X)$，其是原始估计量$\delta(X)$在给定充分统计量$T(x)$下的条件期望：
>
> $$\delta^*(X) = \mathbb{E}[\delta(X)|T(X)]$$
>
> Rao-Blackwell定理指出，这个新估计量$\delta^*(X)$仍然是θ的无偏估计，但方差不多于(通常严格小于)原始估计量$\delta(X)$的方差。

基于Rao-Blackwell定理的思想，核心是先找一个“不那么好”的无偏估计量，然后再优化。那么，公式(2)就是无偏估计量，定义为指示变量

$$I = \begin{cases}1, & \text{如果前 k 个样本都错误} \\ 0, & \text{否则}\end{cases}$$

这个指示变量的期望为

$$\mathbb{E}[I] = 1\cdot P(\text{前 k 个都错误}) + 0\cdot P(\text{其他情况}) = (1-p)^k$$

这个估计量仅利用了前k个样本的信息，方差很大。

现在基于Rao-Blackwell定理，需要找到一个充分统计量。令C表示n次实验中成功的次数，显然C是充分统计量。因此，更优的无偏估计量为，

$$\mathbb{E}[I|C=c] = P(I|C=c) = \frac{P(I,C=c)}{P(C=c)}$$

显然，$P(C=c)=\binom{n}{c}p^c(1-p)^{n-c}$。$P(I,C=c)$的含义是前$k$个样本错误，且剩下的$n-k$个样本中有$c$个正确。所以，$P(I,C=c)=(1-p)^k\cdot\binom{n-k}{c}p^c(1-p)^{n-k-c}=\binom{n-k}{c}p^c(1-p)^{n-c}$。综上，

$$\mathbb{E}[I|C=c] = \frac{\binom{n-k}{c}p^c(1-p)^{n-c}}{\binom{n}{c}p^c(1-p)^{n-c}} = \frac{\binom{n-k}{c}}{\binom{n}{c}}$$

最终，Pass@k的无偏估计量为

$$\rho(n,c,k)=1-\frac{\binom{n-k}{c}}{\binom{n}{c}}$$

> 注：虽然两种证明方法的表示形式上略有不同，但是最终的结果是一致的。

四、Pass@k无偏估计的梯度估计器

在上一节中，已经得到了Pass@k的无偏估计，

$$\text{Pass@k}=\mathbb{E}_{x_1,\ldots,x_n\sim p(x|\theta)}[\rho(n,c,k)]$$

两边同时对θ求梯度，有

$$\nabla_\theta\text{Pass@k}=\nabla_\theta\mathbb{E}_{x_1,\ldots,x_n\sim p(x|\theta)}[\rho(n,c,k)]$$

基于策略梯度定理，将梯度算子移到期望内，

$$\begin{aligned}\nabla_\theta\text{Pass@k}&=\mathbb{E}_{x_1,\ldots,x_n\sim p(x|\theta)}[\rho(n,c,k)\nabla_\theta\log p(x_1,\ldots,x_n|\theta)]\\&=\mathbb{E}\left[\rho(n,c,k)\sum_{i=1}^n\nabla_\theta\log p(x_i|\theta)\right]\\&=\mathbb{E}\left[\sum_{i=1}^n\rho(n,c,k)\nabla_\theta\log p(x_i|\theta)\right]\end{aligned}\tag{6}$$

在随机子集视角下，已知

$$\rho(n,c,k)=\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}\subseteq\{1,2,\ldots,n\}}1-\prod_{i\in\mathcal{I}}\Big(1-f(x_i)\Big)\tag{7}$$

将公式(7)代入到公式(6)中，并将对i的求和移到期望外，有

$$\sum_{i=1}^n\mathbb{E}_{x_i\sim p(x|\theta)}\left[\left(\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}\subseteq\{1,2,\ldots,n\}}\left(1-\prod_{j\in\mathcal{I}}(1-f(x_j))\right)\right)\nabla_\theta\log p(x_i|\theta)\right]\tag{8}$$

> **推论1**. 若c相对于θ和x是常数，那么有$\mathbb{E}_{p(x|\theta)}=[c\nabla_\theta\log p(x|\theta)]=0$。

公式(8)中会对$\binom{n}{k}$个子集进行求和，但是那些不包含x_i的子集对于θ和x_i是常数，依赖推论1可知这部分的期望为0。因此，公式(8)可以进一步简化为

$$\sum_{i=1}^n\mathbb{E}_{x_i\sim p(x|\theta)}\left[\left(\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}\left(1-\prod_{j\in\mathcal{I}}(1-f(x_j))\right)\right)\nabla_\theta\log p(x_i|\theta)\right]\tag{9}$$

仔细观察公式(9)，整个式子可以看作奖励是r_i的策略梯度，其中

$$r_i=\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}\left(1-\prod_{j\in\mathcal{I}}(1-f(x_i))\right)\tag{10}$$

现在通过分情况讨论来简化公式(10)的奖励形式。

若$f(x_i)=1$，那么$\prod_{j\in\mathcal{I}}(1-f(x_i))=0$，奖励$r_i=\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}1$。这个奖励本质是在统计“包含样本$i$的且大小为$k$的子集占比”。要构造这样的子集，必须先选择样本$i$，再从$n-1$个样本中挑选$k-1$个。因此，这样子集的数量为$\binom{n-1}{k-1}$。最终奖励$r_i=\frac{\binom{n-1}{k-1}}{\binom{n}{k}}=\frac{(n-1)!}{(k-1)!(n-k)!}\cdot\frac{k!(n-k)!}{n!}=\frac{k}{n}$。

若$f(x_i)=0$，那么$(1-f(x_i))=1$，奖励

$$
\begin{aligned}
r_i &= \frac{1}{\binom{n}{k}} \sum_{|\mathcal{I}|=k, \mathcal{I}: i \in \mathcal{I}} \left(1 - \prod_{j \in \mathcal{I} \setminus \{i\}} (1 - f_j)\right) \\
&= \frac{1}{\binom{n}{k}} \sum_{|\mathcal{I}|=k, \mathcal{I}: i \in \mathcal{I}} 1 - \frac{1}{\binom{n}{k}} \sum_{|\mathcal{I}|=k, \mathcal{I}: i \in \mathcal{I}} \prod_{j \in \mathcal{I} \setminus \{i\}} (1 - f_j)
\end{aligned} \tag{11}
$$

上式第一项$\frac{1}{\binom{n}{k}} \sum_{|\mathcal{I}|=k, \mathcal{I}: i \in \mathcal{I}} 1 = \frac{k}{n}$。第二项，对于一个包含$i$的子集$\mathcal{I}$，$\mathcal{I} \setminus \{i\}$是一个除了$i$以外从$n-1$个样本中挑选出$k-1$个样本的子集。$\prod_{j \in \mathcal{I} \setminus \{i\}} (1 - f_j)$当且仅当全部由错误样本构成时才为1，否则为0。所以，第二项的含义是：有多少个大小为$k-1$的全部错误子集可以从$n-1$(不包含样本i)个样本中构造出来。在$n-1$个样本中，有c个正确，$n-1-c$个错误。那么，从$n-1-c$个样本中挑选出$k-1$个组成全部错误子集，有$\binom{n-1-c}{k-1}$。合并两项，有

$$
\begin{aligned}
r_i &= \frac{1}{\binom{n}{k}} \left(\binom{n-1}{k-1} - \binom{n-1-c}{k-1} \right) \\
&= \frac{\binom{n-1}{k-1}}{\binom{n}{k}} \left(1 - \frac{\binom{n-1-c}{k-1}}{\binom{n-1}{k-1}} \right) \\
&= \frac{k}{n} \left(1 - \frac{\binom{n-1-c}{k-1}}{\binom{n-1}{k-1}} \right) \\
&= \frac{k}{n} \cdot \rho(n-1, c, k-1)
\end{aligned} \tag{12}
$$

综上，

$$
\begin{aligned}
&\nabla_\theta \text{Pass@k} = \mathbb{E}_{x_1, x_2, \ldots, x_n \sim p(x|\theta)} \left[\sum_i^n r_i \nabla_\theta \log p(x_i|\theta) \right], \\
&\text{where} \quad r_i = \begin{cases} \frac{k}{n} & \text{if } f(x_i) = 1 \\ \frac{k}{n} \cdot \rho(n-1, c, k-1) & \text{if } f(x_i) = 0 \end{cases}
\end{aligned} \tag{13}
$$

直观理解。负样本的奖励相当于在奖励$\frac{k}{n}$的基础上添加了一个潜力系数$\rho(n-1, c, k-1)$。该系数衡量加入当前样本错误，但从余下$n-1$个样本中挑选出$k-1$子集时包含正确答案的潜力。

五、方差缩减

为了降低奖励的方差，需要寻找合适的基线。

5.1 经典LOO基线

在策略梯度中，一种简单且有效的基线是Leave-One-Out(LOO)。其核心思想是，对于当前样本x_i的奖励r_i，使用的基线b_i是除了它自己以外的所有其他样本奖励的均值。即

$$
r_i' = r_i - b_i \quad \text{where} \quad b_i = \frac{1}{n-1} \sum_{j=1, j \neq i}^n r_j \tag{14}
$$

r_i'无偏的前提是基线b_i的计算与当前样本x_i无关，这样就能在求梯度时被视为常数。即

$$
\nabla_\theta \mathbb{E}[r_i'] = \nabla_\theta \mathbb{E}[r_i - b_i] = \nabla_\theta \mathbb{E}[r_i] - \nabla_\theta \mathbb{E}[b_i] = \nabla_\theta \mathbb{E}[r_i] - 0
$$

梯度的期望保持不变，估计仍然是无偏的。通过减去一个与当前奖励r_i高度相关的基线，有效地减小了r_i'的波动范围，从而降低了方差。

5.2 $\mathrm{Pass@k}$的无偏基线

经典LOO有偏。经典的LOO基线无法直接套用到为$\mathrm{Pass@k}$推导出的奖励r_i。回顾公式(13)，奖励r_i依赖于总正确数c，而$c=\sum_{j=1}^{n} f(x_j)$，这意味着r_i的计算本身包含了所有样本的信息。若使用$b_i=\frac{1}{n-1}\sum_{j\neq i} r_j$作为基线，由于每个$r_j$都依赖完整的样本集合，那么间接的依赖于$x_i$，从而引入了偏差。

因此，可以考虑构造一个“反事实”的奖励$r_j^{(-i)}$。假设从样本集合中去除x_i，则得到一个新的集合$\{x_1,\ldots,x_{i-1},x_{i+1},\ldots,x_n\}$，这个集合包含$n-1$个样本，包含$c-f(x_i)$个正确样本。根据公式(13)，得到奖励为

$$r_j^{(-i)}=\begin{cases}\frac{k}{n-1} & \text{if } f(x_j)=1\\ \frac{k}{n-1}\cdot\rho(n-2,c-f(x_i),k-1) & \text{if } f(x_j)=0\end{cases}\quad(15)$$

基于$r_j^{(-i)}$就可以构造出不包含x_i的基线，

$$b_i=\frac{1}{n-1}\sum_{j=1,j\neq i}^{n} r_j^{(-i)}\quad(16)$$

那么最终带基线的奖励为

$$r_i^{(\mathrm{loo})}=r_i-b_i\quad(17)$$

5.3 $\mathrm{Pass@(k\text{-}1)}$作为基线

除了上一小节基于经典LOO改进的基线外，另一种缩减方差的方式是考虑“边际贡献”。基于包含x_i的n个样本集合估计的$\mathrm{Pass@k}$，减去不包含x_i的$n-1$个样本集合估计的$\mathrm{Pass@(k\text{-}1)}$。这个差值相当于衡量了样本$x_i$对$\mathrm{Pass@k}$的贡献，

$$r_i^{(\mathrm{loo}-1)}=\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}\max_{j\in\mathcal{I}} f(x_j)-\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}\max_{b\in\mathcal{I}\setminus\{i\}} f(x_b)\quad(18)$$

公式(18)的第一项就是公式(10)中r_i定义，而第二项可以看作是一个基线。进一步推导一下公式(18)，

$$r_i^{(\mathrm{loo}-1)}=\frac{1}{\binom{n}{k}}\sum_{|\mathcal{I}|=k,\mathcal{I}:i\in\mathcal{I}}\left(\max_{j\in\mathcal{I}} f(x_j)-\max_{b\in\mathcal{I}\setminus\{i\}} f(x_b)\right)\quad(19)$$

在二元奖励下，$\left(\max_{j\in\mathcal{I}} f(x_j)-\max_{b\in\mathcal{I}\setminus\{i\}} f(x_b)\right)$当且仅当$\mathcal{I}$集合成功且$\mathcal{I}\setminus\{i\}$失败时为1，否则为0。分情况讨论下。

若$f(x_i)=0$，$r_i^{(\mathrm{loo}-1)}$必为0。

若$f(x_i)=1$，相当于从$n-c$个错误样本中挑选$k-1$个样本，因此分子为$\binom{n-c}{k-1}$。则$r_i^{(\mathrm{loo}-1)}=\frac{\binom{n-c}{k-1}}{\binom{n}{k}}=\frac{k}{n}\cdot\frac{\binom{n-c}{k-1}}{\binom{n-1}{k-1}}=\frac{k}{n}\cdot[1-\rho(n-1,c-1,k-1)]$。

综上，

$$r_i^{(\mathrm{loo}-1)}=\begin{cases}\frac{k}{n}\cdot[1-\rho(n-1,c-1,k-1)] & \text{if } f(x_i)=1\\ 0 & \text{if } f(x_i)=0\end{cases}\quad(20)$$

直观理解。负样本不给予任何奖励。正样本有一个基础分数$\frac{k}{n}$，$1-\rho(n-1,c-1,k-1)$则是表示在没有x_i(失去一个正样本)的情况下有多大概率失败。

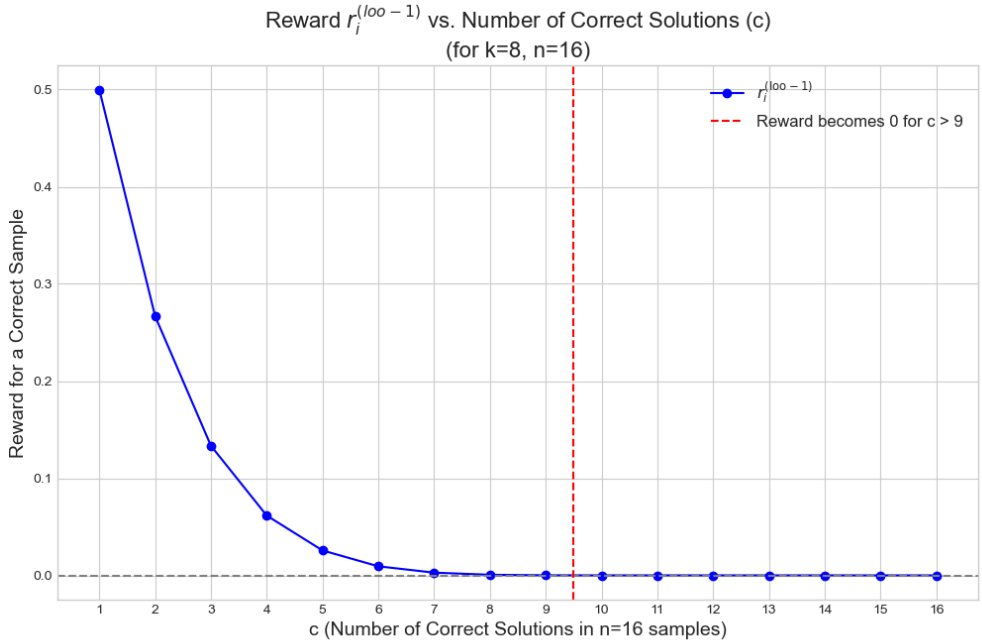
六、$r_i^{(\mathrm{loo}-1)}$与GRPO奖励的分析

1. $r_i^{(\mathrm{loo}-1)}$的直观理解

为了简化分析，不妨假设$n=2k$。那么公式(20)在正样本情况下可以简化为

$$r_i^{(\mathrm{loo}-1)}=\frac{1}{2}\cdot\frac{\binom{2k-c}{k-1}}{\binom{2k-1}{k-1}}\qquad(21)$$

绘制一下$r_i^{\mathrm{loo}-1}$随c变化的曲线



可以看到，随着c的增加，单个正样本的奖励值急剧下降。当$c>k+1$时，即使是正样本，其奖励值也为0。直观的理解，准确率越低的问题($\frac{c}{n}$越小)，单个正样本获得的奖励越大。当准确率超过($\frac{k+1}{n}$)时，正样本也不会获得奖励，即该问题没有必要继续优化。

2. 与GRPO的关系

计算$r_i^{(\mathrm{loo}-1)}$时，同一个输入需要采样n个输出来计算奖励。显然，这与GRPO非常相似。那么在采样n个样本中有c个正样本的假设下，均值为$\mu=\frac{c}{n}$，标准差为$\frac{\sqrt{c(n-c)}}{n}$。

对于$f(x_i)=1$的情况下，有

$$r_i^{\mathrm{grpo}}=\frac{1-\frac{c}{n}}{\frac{\sqrt{c(n-c)}}{n}}=\sqrt{\frac{n-c}{c}}$$

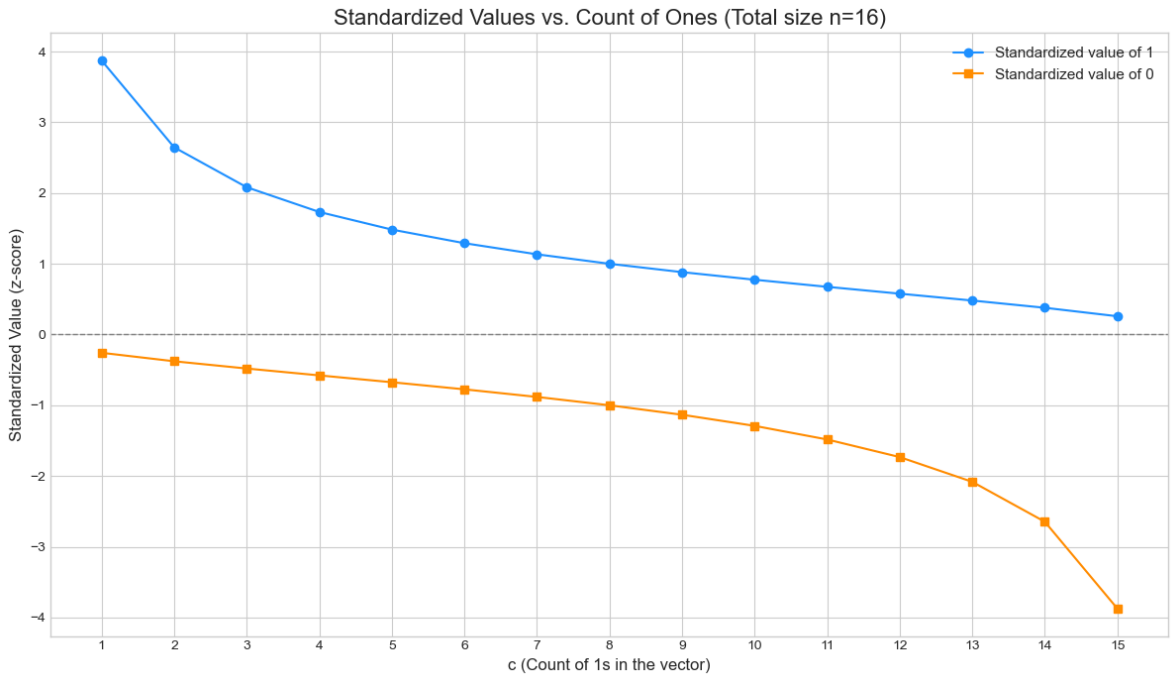
对于$f(x_i)=0$的情况下，有

$$r_i^{\mathrm{grpo}}=\frac{0-\frac{c}{n}}{\frac{\sqrt{c(n-c)}}{n}}=-\sqrt{\frac{c}{n-c}}$$

即

$$r_i^{\mathrm{grpo}}=\begin{cases}\sqrt{\frac{n-c}{c}} & \text{if } f(x_i)=1\\ -\sqrt{\frac{c}{n-c}} & \text{if } f(x_i)=0\end{cases}\qquad(22)$$

同样绘制一下r_i^{grpo}随c变化的曲线



可以看到正样本的情况下，当$c=1$时r_i^{grpo}接近4，随着c的增长r_i^{grpo}持续下降至接近于0。对于负样本，当$c=1$是r_i^{grpo}接近于0，随着c的增长r_i^{grpo}持续下降于接近于-4。

对于正样本，r_i^{grpo}和$r_i^{(\text{loo}-1)}$都是随着c的增加快速下降。因此，这启发了一个合理的假设是GRPO其实已经在优化Pass@k。

七、实现

7.1 $\rho(n,c,k)$的实现

数值稳定。整个方法的核心是$\rho(n,c,k)=1-\frac{\binom{n-c}{k}}{\binom{n}{k}}$，那么如何高效的计算$\frac{\binom{n-c}{k}}{\binom{n}{k}}$是实现整个方法的核心。如果直接计算会涉及到阶乘，为了避免这种情况需要推导出一个更合理的计算形式。

$$\frac{\binom{n-c}{k}}{\binom{n}{k}}=\frac{\frac{(n-c)!}{k!(n-c-k)!}}{\frac{n!}{k!(n-k)!}}=\frac{\frac{(n-c)!}{(n-c-k)!}}{\frac{n!}{(n-k)!}}$$

由于$\frac{n!}{(n-k)!}=n\cdot(n-1)\cdots(n-k+1)$和$\frac{(n-c)!}{(n-c-k)}=(n-c)\cdot(n-c)\cdots(n-c-k+1)$均有$k$项。因此，得到连乘形式

$$\frac{\binom{n-c}{k}}{\binom{n}{k}}=\prod_{i=0}^{k-1}\frac{n-c-i}{n-i}$$

通过上面的连乘形式可以避免上溢的问题。但如果k太大，连乘可能会导致下溢，从而丢失精度。因此，利用对数空间将乘法转换为加法

$$\log\frac{\binom{n-c}{k}}{\binom{n}{k}}=\log\prod_{i=0}^{k-1}\frac{n-c-i}{n-i}=\sum_{i=0}^{k-1}[\log(n-c-i)-\log(n-i)]\quad(23)$$

边界情况。

若$k=0$，则$\rho(n,c,0)=0$。

若$n-c<k$，意味着错误样本数少于要抽取的数量k，所以不可能抽出一个全是不合格的子集。因此，$\binom{n-c}{k}=0$，则$\rho(n,c,k)=1$。

若$k>n$或$c>n$或$k<0$等，无效输入，返回错误。

最终代码。

```
def rho(n: int, c: int, k: int) -> float:
    if k == 0:
        return 0.0

    if n < k or c > n:
        raise ValueError(f"输入无效: n={n}, c={c}, k={k}. "
                         "必须满足 n >= k 和 n >= c。")

    if n - c < k:
        return 1.0

    i = np.arange(k)
    log_numerator = np.sum(np.log(n - c - i))
    log_denominator = np.sum(np.log(n - i))
    log_prob_all_fail = log_numerator - log_denominator
    prob_all_fail = np.exp(log_prob_all_fail)

    return 1.0 - prob_all_fail
```

八、实验

1. 实验设置

基础模型：Qwen2.5-3B

训练集：DAPO-Math-17k

评测集：AIME2024、AMC、MATH、Minerva、Olympiad

评测指标：Pass@4

关键参数：同一个问题采样8个响应，k设置4

2. 基线

Naive：直接使用0和1二元奖励；

$\mathrm{GRPO+}$：仅保留GRPO中正样本的奖励，负样本奖励设置为0，用于验证正样本奖励递减就是在优化$\mathrm{Pass@k}$；

PKPO：使用$r_i^{(\mathrm{loo}-1)}$作为奖励；

3. 结果

	AMIE2024	AMC	MATH	Minerva	Olympiad
Naive	0.166	0.518	0.754	0.356	0.426
GRPO+	0.200(+0.034)	0.469(-0.049)	0.790(+0.036)	0.367(+0.011)	0.431(+0.005)
PKPO	0.166(+0.0)	0.566(+0.048)	0.814(+0.06)	0.404(+0.048)	0.451(+0.025)